



UCI Samueli
School of Engineering

UCI ROCKET PROJECT

ROCKET.ENG.UCI.EDU

Krinsky's Guide to Rocket GUI

Spring 2025

Last Updated: 5-13-2025
Original Slides [Here](#)



Installation



UCI Samueli
School of Engineering

Follow the instructions in the rocket2-webservice-gui README.md for how to install:

<https://github.com/UCI-Rocket-Project/rocket2-webservice-gui>



Design Choices



UCI Samueli
School of Engineering

Our System Requirements and Choices

- Easy to build and develop with multiple people
 - Chose Python for easy development and familiarity with libraries
 - Chose React for its popularity and easiness to learn
 - Put everything in Docker containers to standardize versioning and avoid “well it works on my machine” problems
- Can communicate quickly with our electronics over TCP/IP
 - Used Python socket connections to handle TCP communications(only real choice)
 - Spawns threads to handle DB tasks and GUI requests to keep focus on receiving packets
- Quickly updates the state of our control screen
 - Used HTTP endpoints to quickly pass states to the control screens (should be websockets for faster speed but was not necessary for the speed we wanted)
- Can save the state to a database
 - Chose Dockerized SQL database for familiarity



High Level Design



UCI Samueli
School of Engineering

- **Python Web Service**
 - Establishes connections with AV systems to receive states and send commands
 - Stores most recent rocket state to give to GUI when it asks for it
 - Compares the rocket's solenoid states with the expected rocket state to determine when it needs to send an update command
 - Receives commands from the GUI and transmits them to the AV systems
 - Handles all DB requests from the GUI
- **React Frontend**
 - Sends requests to the web service to receive state and send commands
 - Displays state information about the AV system and DB to the user
 - Takes inputs from the user and transmits their commands to the web service
- **SQL Database**
 - Reliably stores packets sent to it by the web service
 - Allows querying of the database by the web service

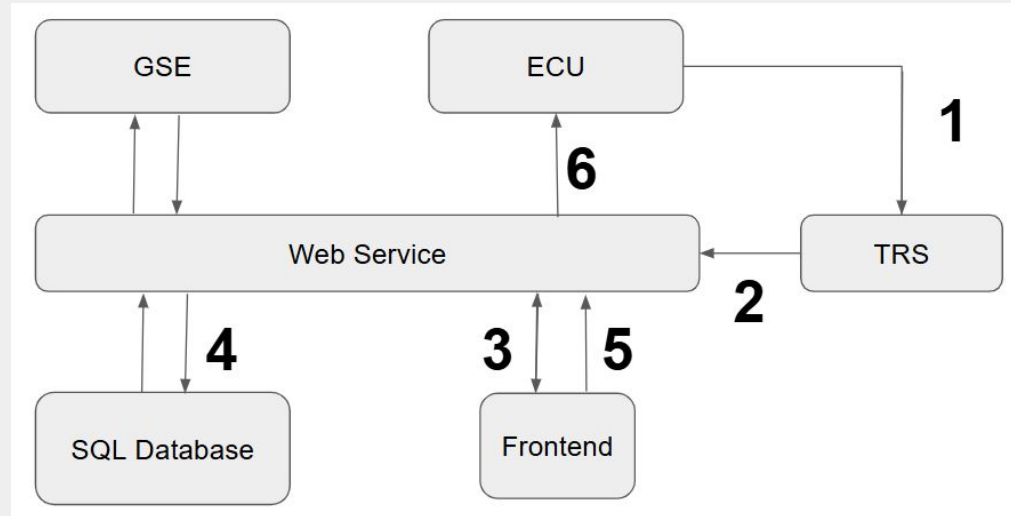


Packet Flow



UCI Samueli
School of Engineering

1. ECU sends state packet over radio to TRS
2. TRS forwards packet over sockets to Web Service
3. Frontend requests and receives packet over HTTP from Web Service
4. Web Service saves packet to the SQL DB
5. Frontend sends a solenoid command over HTTP to Web Service
6. Web service forwards solenoid command over sockets to ECU





Web Service Processes



UCI Samueli
School of Engineering

1. Create threads for each AV system and open socket connections at their IP and port
2. When we receive data on the socket:
 - a. decode the data into a packet using the agreed packet format
 - b. update the state on the web service
 - c. Compare the packet's solenoid states with the expected states to determine if they are aligned
 - i. If they aren't aligned, spawn a new thread to send a command over the corresponding socket to match the solenoids
 - d. Create a new thread to handle saving it to the DB in so we can receive more data before we have the last packet saved
3. Startup flask web service to allow the GUI to fetch state and send commands over HTTP
 - a. If they ask for an AV system's state
 - i. Return that system's state from the web service
 - b. If they send a command to the AV system
 - i. Update the expected state for the solenoid
 - ii. Send a command over the socket to the AV system

```
webservice > server.py > ...  
406 if __name__ == "__main__":  
407     gse_listening_thread = Thread(  
408         target=start_system_listening,  
409         args=(  
410             (gse_ip, gse_port),  
411             gse_connection_lock,  
412             GSE_DATA_LENGTH,  
413             "<L????????????????ffffffffffffffff",  
414             handle_update_gse_state,  
415             "GSE",  
416         ),  
417     )  
418     gse_listening_thread.daemon = True  
419     gse_listening_thread.start()
```

The GSE listener thread

```
webservice > server.py > ...  
193 @app.route("</system_name>/state", methods=["GET"])  
194 > def get_state(system_name): ...  
206  
207  
208 @app.route("</system_name>/state/<switch_name>/<new_state>")  
209 > def set_solenoid(system_name, switch_name, new_state): ...  
241
```

The HTTP endpoints on the webservice



How the Frontend Works



- App.jsx is the top level object that gets displayed and handles communication with the webservice and the storage of the rocket state
 - On startup, it sets an interval(loop) to fetch the states of the AV systems using the HTTP endpoints
 - When it receives a state, it updates its own storage of the state which triggers a rerender of the page
- Depending on which page you click on, the App.jsx will render it and provide the state to that page using the `<RocketState.Provider>`
 - The pages can access the variables passed in the `RocketState` with the `useContext` function
- `webservice.js` uses `axios` library to send commands and get states from the web service over HTTP

```
gui > src > App.jsx > ...
14  export function App() {
165      return (
166          <RocketState.Provider
167              value={{
168                  solenoids,
169                  tcs,
170                  pts,
171                  igniters,
172                  misc,
173              }}>
174              <Routes>
175                  <Route
176                      path="/"
177                      element={<DashboardPage />}
178                  />
179              </Routes>

```

RocketState provider passing in solenoids and other states to the control screen pages



How does Docker Work



- Dockerfiles create a virtual computer inside of your computer to run stuff exactly the same way regardless of what computer it is being run on
 - Inside a Dockerfile you define a base image which is equivalent to like “linux computer with python installed”
 - Then you define all of the commands and files you want to put into it so it can run code for you
- Docker-compose files create virtual computer networks to allow docker containers to communicate with one another in a simpler way
 - Inside docker-compose file you tell it which docker containers to use, which ones can communicate with one another, and what files, and ports they have access to on your computer
 - A docker container can contact another container by using the other containers name as the IP address(assuming you put them on the same network)
- In order to run a docker compose file, you must have its docker containers built already using make build

```
webservice > Dockerfile
1  # Use an official Python runtime as a parent image
2  FROM python:3.12.0a4
3  # Copy the current directory contents into the container at /app
4  COPY . /app
5  # Install any needed packages specified in requirements.txt
6  RUN pip install --no-cache-dir -r requirements.txt
7  # Make port 8000 available to the world outside this container
8  EXPOSE 8000
9  # Run app.py when the container launches
10 CMD ["python3", "server.py"]
```

Webservice Dockerfile defining the image, ports and startup command

```
docker-compose-dev.yaml
3  services:
14     webservice:
15         image: webservice
16         ports:
17             - "8000:8000"
18         volumes:
19             - ./webservice:/app
20         networks:
21             - basic_net
```

Docker Compose file defining the webservice's network and available ports



Docker Commands



UCI Samueli
School of Engineering

- **docker compose -f docker-compose-prod.yaml down**
 - Deletes existing docker containers
 - You need to run this when you make changes to the DB because rebuilding containers do not reinitialize the DB
- **make build**
 - Creates or rebuilds all docker containers using the Dockerfiles provided in the repo
 - Run this when you make changes to any of the files that are not in the “volumes” section of the docker compose, as the changes need to be moved from your computer into the virtual one
 - Ex: gui/public is not mounted in the volumes of gui but gui/src is
 - Mainly needed for when you npm install something new(gui/node_modules shouldn't be in volumes)
- **make run**
 - Starts up the docker-compose-prod.yaml containers and creates a network for them to communicate
 - Uses IP addresses of AV systems to connect to them
- **make run_dev**
 - Starts up the docker-compose-dev.yaml containers and creates a network for them to communicate
 - Uses localhost for AV systems to connect to fake_rocket instead



MK's Rocketry for Dummies



UCI Samueli
School of Engineering

MK's Rocketry for Dummies

April 29, 2023



UCI Rocket Project

MK's Rocketry for Dummies

- Methalox fuel
- Cold Flows
- Static Fire
- Vertical Test Fire
- Pressure Transducers (PTs)
- Thermocouples (TCs)

Github

<https://github.com/UCI-Rocket-Project>